

Nurvai Core Engineering Challenge: The Embodied Data Alchemist (Senior Version)

1. Visión General del Reto

En **Nurvai** construimos la infraestructura de datos de alta fidelidad que impulsará los modelos de Inteligencia Artificial Corporeizada (Embodied AI) del futuro.

Tu objetivo es construir un pipeline automatizado de post-procesamiento que tome datos crudos de una sesión de recolección egocéntrica, resuelva la desalineación de los sensores y enriquezca el dataset con características espaciales necesarias para entrenar modelos **VLA (Vision-Language-Action)**.

Tienes un límite de **3 días** para entregar tu solución a partir de la recepción de este documento.

2. Los Insumos (Input Data)

Se te proporcionarán dos archivos crudos de una misma sesión:

1. **video.mp4**: Grabación de la cámara principal (30 FPS).
 2. **imu.csv**: Lecturas del sensor inercial a alta frecuencia (~574 Hz) con las columnas:
 - a. timestamp_ns, gyro_x, gyro_y, gyro_z, accel_x, accel_y, accel_z
 3. **pts.csv**: Puntos de alineación, marcas de línea de tiempo de video(~30 Hz).
 - a. frame_index, timestamp_ns
-

3. El Desafío (Tus Tareas)

Debes diseñar y programar un pipeline en **Python** estructurado en tres fases:

Fase 1: Alineación Temporal y Fusión de Sensores

Los sensores corren a frecuencias distintas y no están coordinados en origen. Debes implementar una solución matemática para alinear temporalmente el flujo de video con la

telemetría del IMU, asegurando que cada frame de video cuente con su correspondiente estado inercial exacto.

Fase 2: Extracción de Características Espaciales (Ego-Tracking)

Aplica modelos de visión por computadora modernos para extraer del video la posición y tracking de las manos del operador en el espacio. Tu pipeline debe ser robusto ante oclusiones y cambios bruscos de iluminación.

Fase 3: Estructuración y Segmentación de Acción

Agrupar y etiquetar los datos en bloques temporales (*action chunks*). Diseña una lógica o heurística para determinar de manera automática los momentos de "actividad" frente a los de "estatismo" a lo largo de la sesión.

4. Los Entregables

El repositorio privado de GitHub debe contener:

1. **Entorno Dockerizado (**Dockerfile**)**: El pipeline completo debe ejecutarse dentro de un contenedor optimizado con soporte para GPU (CUDA). El evaluador ejecutará el proceso mediante una interfaz de línea de comandos (CLI) apuntando a los archivos de entrada.
2. **Dataset Enriquecido (**.jsonl**)**: Un archivo JSON Lines estructurado de forma eficiente para bases de datos NoSQL (MongoDB), donde cada entrada representa un frame unificado con su IMU interpolado, tracking espacial de extremidades y metadatos de acción.
3. **Utility de QA (**output_qa.mp4**)**: Un script secundario que genere un video de verificación donde se rendericen, sobre el video original, los keypoints extraídos y la telemetría sincronizada en tiempo real.